

CloudStore SDK for Python (Quickstart)

The CloudStore SDK for Python enables programmatic access to CloudStore buckets and objects. This guide covers client initialization, file upload with custom metadata, and exception handling.

Prerequisites

- Python **3.9 or later**
- A valid CloudStore API key with **Storage:Write** permissions

1. Installation

Install the official SDK package using **pip**:

```
Bash
pip install cloudstore-sdk
```

2. Configure Environment Variables

The SDK automatically reads credentials and region configuration from the runtime environment. Export the following variables in your shell or **.env** file:

```
Bash
export CLOUDSTORE_API_KEY="csk_live_09f8e7d6c5b4a3a2"
export CLOUDSTORE_REGION="us-east-1"
```

Security Note: Do not hardcode API keys directly into source files or version control. Use environment variables or a secrets manager.

3. Upload an Object with Metadata

Use the **StorageClient.objects.upload()** method to transfer a local file to a designated bucket.

Code Example

```
Python
import os
import sys
from cloudstore import StorageClient
from cloudstore.exceptions import (
    AuthenticationError,
    BucketNotFoundError,
    StorageError,
```

```

)

def upload_document(bucket: str, destination_key: str, local_file_path:
str):
    # Initialize client; automatically discovers environment credentials
    client = StorageClient()

    try:
        response = client.objects.upload(
            bucket_name=bucket,
            key=destination_key,
            file_path=local_file_path,
            metadata={
                "department": "finance",
                "classification": "confidential"
            }
        )

        print("Upload successful.")
        print(f"ETag: {response.etag}")
        print(f"Version ID: {response.version_id}")
        print(f"Timestamp: {response.uploaded_at}")
        return response

    except AuthenticationError:
        print("Error: Invalid or expired API credentials.",
file=sys.stderr)
    except BucketNotFoundError:
        print(f"Error: Bucket '{bucket}' was not found in region
{client.region}.", file=sys.stderr)
    except StorageError as err:
        print(f"Operational error during upload: {err.message}",
file=sys.stderr)

if __name__ == "__main__":
    upload_document(
        bucket="prod-reports-archive",
        destination_key="2026/q3/quarterly_audit.pdf",
        local_file_path="./quarterly_audit.pdf"
    )

```

Method Reference: `client.objects.upload()`

Request Parameters

Parameter	Type	Required	Description
<code>bucket_name</code>	<code>str</code>	Required	The target bucket name. Must conform to standard naming rules (lowercase alphanumeric and hyphens).
<code>key</code>	<code>str</code>	Required	The destination path / identifier within the bucket (for example, <code>logs/app.log</code>).
<code>file_path</code>	<code>str</code>	Required	Path to the local file to upload.
<code>metadata</code>	<code>dict[str, str]</code>	Optional	User-defined key-value tags associated with the object. Maximum 10 key-value pairs.

Return Value (`UploadResponse`)

Attribute	Type	Description
<code>etag</code>	<code>str</code>	MD5 checksum hash identifying the specific representation of the uploaded object.
<code>version_id</code>	<code>str</code>	Unique version ID assigned by the bucket (if versioning is enabled).
<code>uploaded_at</code>	<code>str</code>	ISO-8601 UTC timestamp of completed ingestion.

Common Exceptions

Exception Class	HTTP Equivalent	Recommended Remediation
AuthenticationError	401 / 403	Verify that <code>CLOUDSTORE_API_KEY</code> is loaded and has write policies attached.
BucketNotFoundError	404	Confirm the bucket name is spelled correctly and exists in the configured region.
StorageError	500 / 503	Base SDK exception for socket timeouts or service outages. Retry with exponential backoff.